DATA STRUCTURES AND ALGORITHMS

Unit I: Arrays and Lists

In this unit

1. Basics of data structures and their classification **2.** Algorithms and complexity **3.** Arrays and their operations

4. Two-dimensional arrays and memory allocation **5.** Sparse matrices **6.** Applications and string handling

1. Introduction to Data Structures

1.1 What is a data structure?

A **data structure** is a systematic way of arranging a collection of data items in a computer's memory so that they can be stored, accessed and modified efficiently. Arrays, linked lists, stacks and queues are common examples.

Data structures sit at the core of almost every area of computing, including operating systems, compiler design, artificial intelligence and computer graphics.

1.2 Why do we need them?

Data rarely exists in isolation. A data structure captures how one item relates to another and decides how those items are laid out in memory. Depending on the problem, that relationship can be simple or complex:

- **Simple (linear):** the months of a year, where each item has one predecessor and one successor.
- **Complex (non-linear):** a set of famous monuments grouped by country, region and city, where items branch into several related items.

A good choice of structure lets us store, search and analyse data in an organised and logical way.

1.3 Data versus information

Aspect	Data	Information
Meaning	Raw, unprocessed facts about an event, object or idea	Data that has been processed and given meaning
Form	Plain numbers, text and symbols	Organised, interpreted results
Usefulness	Little value on its own	Supports conclusions and decisions
Example	35, 41, 29, 38	"The average temperature this week was 35.75 °C"

2. Classification of Data Structures

Data structures are broadly split into **primitive** and **non-primitive** types. Non-primitive structures are further divided into linear and non-linear ones, as shown below.

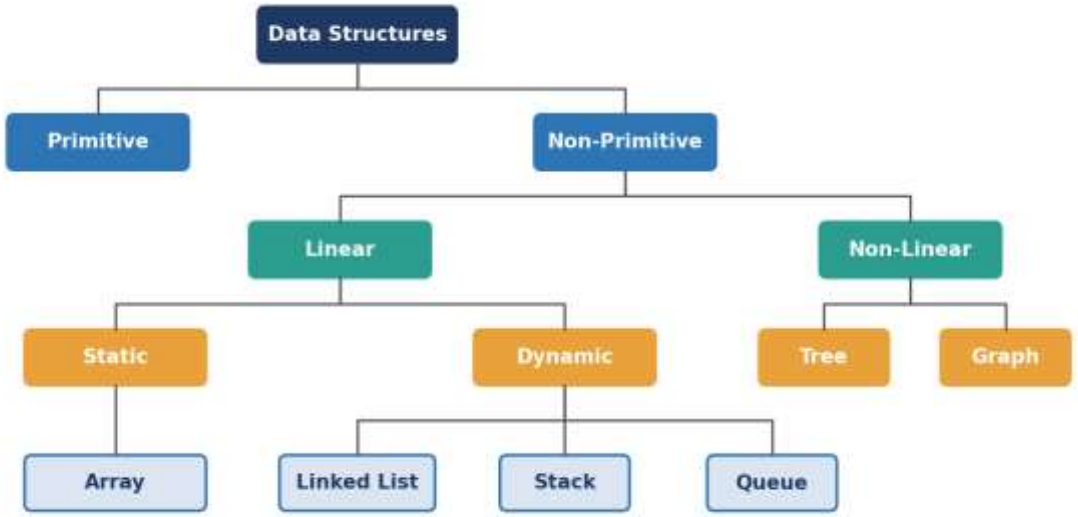


Figure 1: *Classification of data structures*

2.1 Primitive data structures

Primitive structures are the basic building blocks supplied by the language itself. They map directly onto machine-level representations, so their size can differ from one system to another. Programmers use them to declare variables.

Category	Java type	Typical size	Example
Logical	`boolean`	true / false	`boolean ok = true;`
Character	`char`	16 bits	`char grade = 'A';`
Integer	`byte`, `short`, `int`, `long`	8, 16, 32, 64 bits	`int age = 20;`
Floating-point	`float`, `double`	32, 64 bits	`double pi = 3.14;`

> **Note**
> `String` is **not** a primitive type. It is an object (a non-primitive type) built on top of characters, which is why the classification above places strings and arrays under non-primitive data.

2.2 non-primitive data structures

These are more elaborate structures that are derived from primitive types. They are either linear or non-linear.

Linear data structures

In a linear structure the elements form a single sequence. Every element (except the first and last) has exactly one predecessor and one successor, so the whole structure can be visited in one pass from start to end. Linear structures are generally easier to implement because the elements sit in an orderly sequence.

Structure	Principle	Key points
Array	Fixed-size collection of same-type items	Stores many values under one name, for example 10 roll numbers in a single array instead of 10 separate variables.
Stack	LIFO: Last In, First Out	The newest item is removed first. Adding is called push and removing is called pop.
Queue	FIFO: First In, First Out	The oldest item is removed first. Insertion at the rear is enqueue; removal from the front is dequeue.
Linked list	Chain of nodes	Each node holds a data value and a reference to the next node.

Non-linear data structures

Here the elements are not placed in a single sequence. An element may be connected to **more than two** others, so the structure cannot be traversed in one straight pass.

Tree. A tree is a hierarchical structure made of nodes joined by parent-child links. It starts at one root node and branches downwards. An organisation chart is a familiar example: in Figure 2, A could be a manager, B and C officers, and the remaining nodes clerks.

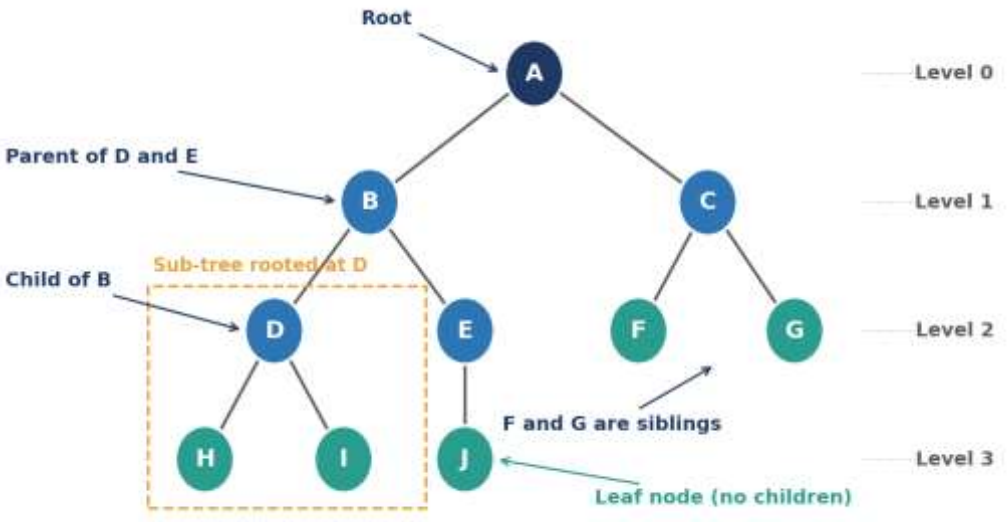


Figure 2: *A tree and its terminology*

Graph. A graph consists of a finite set of **vertices** (nodes that hold data) and **edges** (links that express relationships between vertices). Telephone networks, circuit layouts and social networks are

modelled with graphs. On a social platform, each user is a vertex and each friendship or connection is an edge.

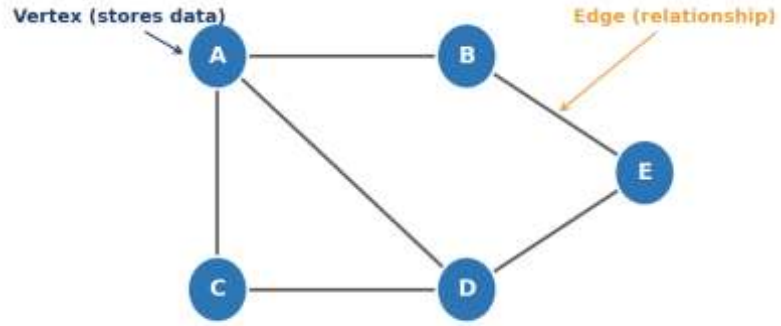


Figure 3: *A graph with five vertices and six edges*

Linear versus non-linear at a glance

Feature	Linear	Non-linear
Arrangement	Sequential, one after another	Hierarchical or networked
Links per element	At most two (previous and next)	Can be more than two
Traversal	Single pass	Needs special traversal methods
Implementation	Relatively simple	More complex
Examples	Array, stack, queue, linked list	Tree, graph

3. Algorithms

An **algorithm** is a finite, ordered set of well-defined steps that solves a specific problem. It is not a program; it is the underlying logic, and it can be written as plain-language steps, a flowchart or pseudocode. The same algorithm may later be coded in any programming language.

3.1 Characteristics of a good algorithm

Property	Meaning
Input	Takes zero or more clearly defined inputs.
Output	Produces at least one well-defined result that matches what the problem requires.
Finiteness	Must stop after a finite number of steps; it cannot run forever.
Language independence	Describes the steps in a form that does not rely on any one programming language.
Unambiguity	Every step, input and output is precise, so it can be interpreted in only one way.

3.2 Time and space complexity

A problem can usually be solved in several ways, so we need a fair method to compare them. Two measures are used most often, both expressed as a function of the input size:

- **Time complexity:** how the running time of the algorithm grows as the input grows.
- **Space complexity:** how much extra memory the algorithm needs while it runs. This matters when memory is limited or when many users share the same system.

3.3 Example: multiplying two numbers

Problem: read two numbers `x` and `y`, multiply them and show the product in `z`.

> **ALGORITHM**
>
> ```
> Step 1: START
> Step 2: Declare integer variables x, y and z
> Step 3: Read the values of x and y
> Step 4: Compute x * y
> Step 5: Store the result in z
> Step 6: Display z
> Step 7: STOP
> ```

4. Arrays

4.1 Concept and definition

An **array** is a collection of elements of the same data type stored in **contiguous memory locations**. Each element is reached through a numeric position called an **index**. Instead of declaring `number0`, `number1`, ... `number99` separately, we declare one array `numbers` and refer to its items as `numbers[0]` to `numbers[99]`.

Because the elements are stored together and share one type, related values can be sorted or searched easily. The lowest memory address holds the first element and the highest address holds the last.

> **Definition**
>
> An array is a fixed-size, ordered collection of same-type elements in contiguous memory, each accessible directly by its index.

4.2 Types of arrays

- **One-dimensional (1-D)** or linear arrays: a single row of elements, accessed by one subscript.
- **Multi-dimensional arrays:** two-dimensional (matrix) arrays and three-dimensional arrays.

4.3 One-dimensional arrays

Two terms are used constantly: an **element** is a single stored item, and an **index** is the numeric position of that item. In Java, the first element is at index 0 and the last is at index `length - 1`.

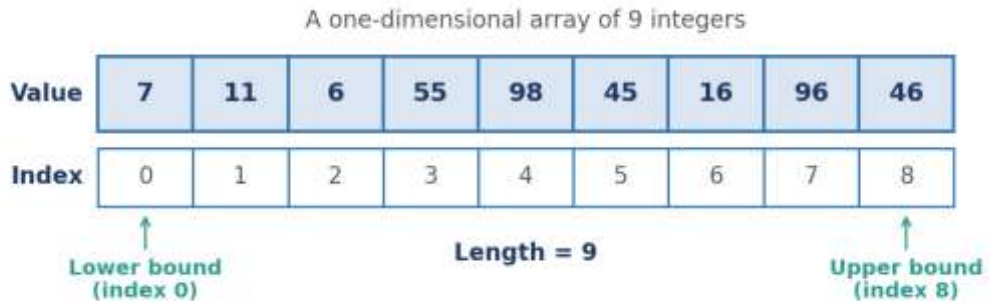


Figure 4: *Elements and indexes of a one-dimensional array*

4.4 Declaring and creating arrays (Java)

Style	Syntax	Example
Declaration only	`dataType[] name;`	`int[] arr;`
Declaration with allocation	`dataType[] name = new dataType[size];`	`int[] arr = new int[5];`
Declaration with values	`dataType[] name = {v1, v2, v3};`	`int[] arr = {10, 20, 30};`
Allocation with values	`dataType[] name = new dataType[]{v1, v2};`	`int[] arr = new int[]{10, 20};`

An alternative placement of brackets, `int arr[];`, is also accepted in Java but the style `int[] arr;` is preferred.

4.5 Basic operations

Five operations are supported on arrays: **traverse**, **insert**, **delete**, **search** and **update**. Their typical costs are summarised below.

Operation	What it does	Time (worst case)
Traverse	Visits every element once	O(n)
Access / update by index	Reads or replaces the value at a known position	O(1)
Search by value (linear)	Checks elements one by one	O(n)
Insert at a position	Shifts later elements right, then stores the new value	O(n)
Delete at a position	Shifts later elements left to close the gap	O(n)

(a) Traversal

Traversal means visiting each element in order, normally with a loop that runs from index 0 to `length - 1`.

```
int[] arr = {10, 20, 30, 40, 50};

for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

(b) Insertion

Because an array has a fixed size and its elements are contiguous, inserting at position `index` requires making room first. Every element from `index` onward moves one place to the right (working from the end), the new value is stored in the gap, and the element count increases by one. The array must have spare capacity, and the index must lie between 0 and the current count.

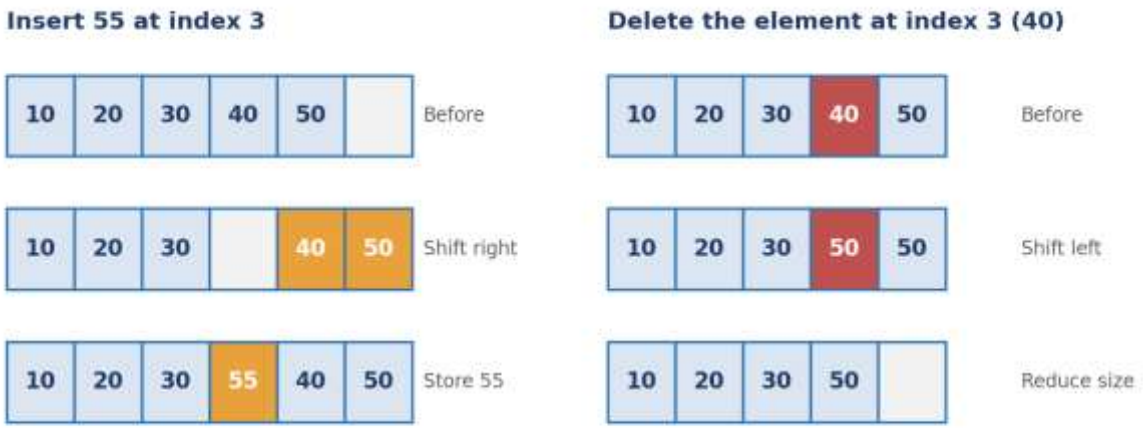


Figure 5: *Insertion (left) and deletion (right) by shifting elements*

JAVA: INSERT 55 AT INDEX 3

```
public class InsertDemo {
    public static void main(String[] args) {
        int[] arr = new int[10];            // capacity of 10
        arr[0] = 10; arr[1] = 20; arr[2] = 30;
        arr[3] = 40; arr[4] = 50;
        int count = 5;                      // elements currently stored
        int pos = 3;                        // where to insert
        int value = 55;                     // what to insert

        if (count == arr.length) {
            System.out.println("Array is full, cannot insert");
        } else if (pos < 0 || pos > count) {
            System.out.println("Invalid position");
        } else {
            for (int i = count; i > pos; i--) {
                arr[i] = arr[i - 1];        // shift right
            }
```

```
            arr[pos] = value;
            count++;
        }
        for (int i = 0; i < count; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
// Output: 10 20 30 55 40 50
```

(c) Deletion

To delete the element at a given index, every element after it moves one place to the left, overwriting the unwanted value. The logical size then decreases by one.

JAVA: DELETE THE ELEMENT AT INDEX 3

```
int[] arr = {10, 20, 30, 40, 50};
int count = arr.length;
int pos = 3;                                    // delete 40

for (int i = pos; i < count - 1; i++) {
    arr[i] = arr[i + 1];                        // shift left
}
count--;

for (int i = 0; i < count; i++) {
    System.out.print(arr[i] + " ");
}
// Output: 10 20 30 50
```

(d) Searching

Searching answers whether a value exists and, if so, where. There are two situations:

- **By index:** the position is known and we want the value. Arrays allow direct access, so this is very fast.
- **By value (linear search):** the value is known and we want its position. Elements are checked one after another until a match is found or the end is reached.

JAVA: TWO WAYS TO SEARCH

```
int[] arr = {10, 20, 30, 40, 50};

// Search by index
System.out.println("Element at index 3: " + arr[3]);

// Search by value (linear search)
int key = 30;
boolean found = false;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == key) {
        System.out.println("Found at index " + i);
        found = true;
        break;
    }
```

```
}
if (!found) {
    System.out.println("Not found");
}
```

(e) Update

Updating replaces the value at a given index. The size of the array stays the same and no shifting is required.

```
int[] arr = {10, 20, 30, 40, 50};
arr[1] = 32;                                    // replace 20 with 32
// Result: 10 32 30 40 50
```

4.6 Sorting an array

Sorting rearranges elements in ascending (smallest to largest) or descending order. Java offers a ready-made method, and we can also write the logic ourselves.

Using the built-in method

```
import java.util.Arrays;

int[] arr = {50, 20, 40, 10, 30};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));   // [10, 20, 30, 40, 50]
```

Bubble sort. This method repeatedly compares neighbouring elements and swaps them whenever they are out of order. After each pass, the largest unsorted value settles at the end of the array, as if it "bubbled up".

JAVA: BUBBLE SORT

```
int[] arr = {50, 20, 40, 10, 30};
int n = arr.length;

for (int pass = 0; pass < n - 1; pass++) {
    for (int j = 0; j < n - 1 - pass; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];              // swap neighbours
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
for (int x : arr) {
    System.out.print(x + " ");
}
```

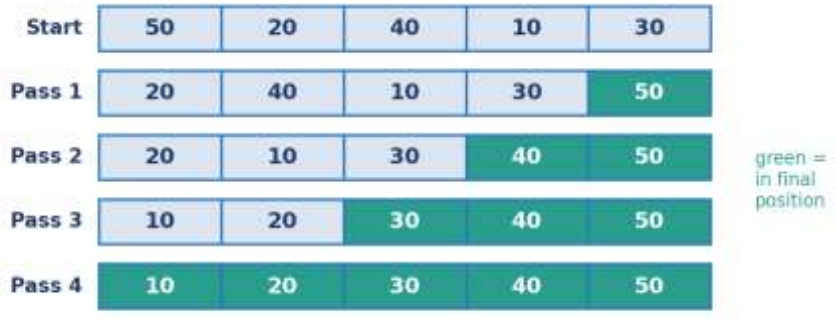


Figure 6: *Bubble sort trace for {50, 20, 40, 10, 30}*

Bubble sort takes O(n^2) time in the worst case, so it is mainly used for teaching and for very small data sets.

4.7 Two-dimensional arrays

A two-dimensional array is an array whose elements are themselves arrays. It stores data in **rows and columns**, like a table, and each element is reached with two indexes: `name[row][column]`.

Purpose	Syntax	Example
Declaration	`dataType[][] name;`	`int[][] matrix;`
Allocation	`dataType[][] name = new dataType[rows][cols];`	`int[][] matrix = new int[2][3];`
Initialisation	`dataType[][] name = {{...}, {...}};`	`int[][] matrix = {{1, 2, 3}, {4, 5, 6}};`

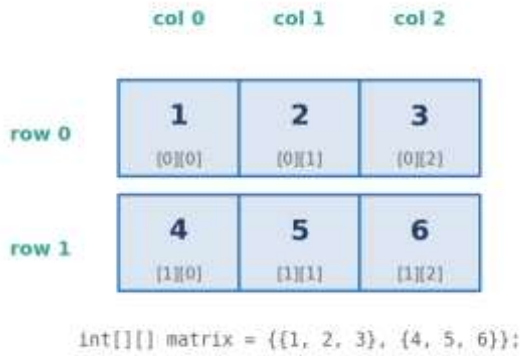


Figure 7: *Layout of a 2 × 3 array with row and column indexes*

Accessing elements:

```
System.out.println(matrix[0][1]);   // 2
System.out.println(matrix[1][2]);   // 6
```

Traversal uses two nested loops: the outer loop walks through rows and the inner loop through the columns of each row.

```
for (int i = 0; i < matrix.length; i++) {          // rows
    for (int j = 0; j < matrix[i].length; j++) {   // columns
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

4.8 Memory allocation

Memory allocation is the process by which a running program obtains memory to hold its variables, arrays and objects. It happens in two ways.

Feature	Static allocation	Dynamic allocation
When decided	At compile time	At run time
Size	Fixed; must be known in advance	Can be chosen according to need
Memory area	Stack (locals) or data segment (globals, statics)	Heap
Release	Automatic when scope or program ends	In C, by the programmer; in Java, by the garbage collector
Trade-off	Fast and simple, but inflexible and may waste space	Flexible and economical, but slightly slower to manage

In **Java**, arrays are always created dynamically with `new` and live on the **heap**. The variable (for example `arr`) is only a reference stored on the **stack**, and it points to the array object on the heap. Python hides these details and handles allocation and garbage collection automatically.

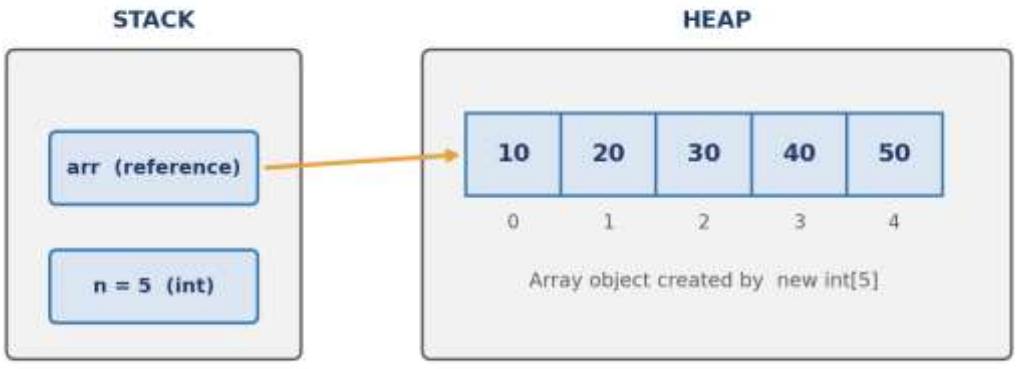


Figure 8: *A Java array: reference on the stack, data on the heap*

4.9 Indexing

Indexing means accessing an element through its position number, written `arrayName[index]`. Java uses **zero-based** indexing: the first element is at index 0 and the last at `length - 1`. Using an index outside this range causes an `ArrayIndexOutOfBoundsException`.

```
int[] arr = {10, 20, 30, 40, 50};
System.out.println(arr[0]);                 // 10  (first)
System.out.println(arr[2]);                 // 30
System.out.println(arr[arr.length - 1]);    // 50  (last)
```

5. Sparse Matrices

A matrix with *m* rows and *n* columns holds m × n values. If the **majority of those values are zero**, the matrix is called a **sparse matrix**. Put differently, the zeros greatly outnumber the non-zero entries.

Example: the matrix below has 4 rows and 5 columns, so 20 positions, but only 6 of them are non-zero.

```
0  0  3  0  4
0  0  5  7  0
0  0  0  0  0
0  2  6  0  0
```

5.1 Why use a special representation?

- **Memory:** storing every zero in a normal 2-D array is wasteful. Keeping only the non-zero elements saves a large amount of space.
- **Speed:** operations can skip the zeros and process only the meaningful entries, which saves time.

Two common representations are the **triplet (array) form** and the **linked-list form**.

5.2 Triplet (array) representation

Each non-zero element is described by three fields: its **row** index, its **column** index and its **value**.

How to build the triplet table:

1. Create a table with three columns (Row, Column, Value) and (number of non-zero elements + 1) rows.
2. In the first row, record the matrix's total rows, total columns and count of non-zero elements.
3. Scan the matrix row by row, left to right (row-major order).
4. For every non-zero element found, add a row to the table with its row index, column index and value.



Figure 9: *Converting a sparse matrix to triplet form*

5.3 Linked-list representation

Here every non-zero element is kept in its own node, and the nodes are chained together. The main benefit over the triplet array is that nodes can be added or removed without shifting other elements, and memory is used only for non-zero values.

Field	Purpose
Row	Row index of the non-zero element
Column	Column index of the non-zero element
Value	The non-zero value itself
Next	Reference (address) of the next node; NULL at the end

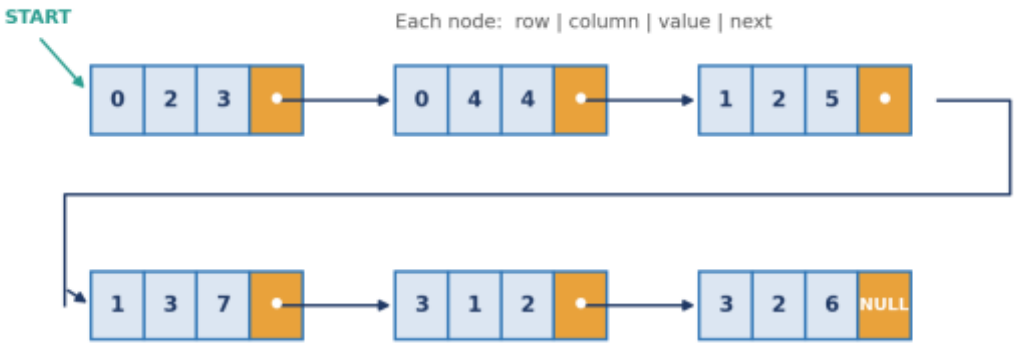


Figure 10: *The same sparse matrix stored as a linked list*

5.4 Operations on sparse matrices

(a) Transpose

The transpose of a matrix swaps its rows and columns, so an entry at (row, column) moves to (column, row). In triplet form, the simple method scans the table once for every column number 0,

1, 2, ... and, each time it finds an entry in that column, writes it into the result with its row and column swapped. This automatically keeps the result sorted by row.

ALGORITHM: SIMPLE TRANSPOSE

```
1. Read the triplet form of the matrix (header: rows, cols, nz).
2. Result header = (cols, rows, nz).
3. Set k = 1.
4. For c = 0 to cols - 1:
       For each entry i in the triplet (from row 1 onward):
           If entry[i].column == c:
               result[k] = (c, entry[i].row, entry[i].value)
               k = k + 1
5. Stop.
```

Applying this to the example matrix gives a 5 × 4 result:

	Row	Column	Value
Header	5	4	6
1	1	3	2
2	2	0	3
3	2	1	5
4	2	3	6
5	3	1	7
6	4	0	4

(b) Addition

Condition: both matrices must have identical dimensions. The idea is to walk through the two triplet tables together, always moving ahead on whichever entry has the earlier position, and to add the values when both matrices have a non-zero entry at the same position.

ALGORITHM: ADD TWO SPARSE MATRICES

```
1. Check that A and B have the same number of rows and columns.
2. Create result table C; copy rows and columns into its header.
3. Let i, j, k be positions in A, B and C. Set all three to 1.
4. While both A and B still have entries:
     a. If A[i].row  <  B[j].row  (or same row and smaller column):
            copy A[i] into C[k];  i++, k++
     b. Else if B[j] comes earlier in the same way:
            copy B[j] into C[k];  j++, k++
     c. Else (same row and column):
            sum = A[i].value + B[j].value
            if sum != 0: store (row, column, sum) in C[k];  k++
            i++, j++
5. Copy any entries left over in A or B into C.
6. Write the count of entries (k - 1) into the header of C.
```

(c) Multiplication

Condition: the number of columns of the first matrix must equal the number of rows of the second. If A is m × n and B is n × p, the product is m × p.

1. Find the **transpose of the second matrix**. This makes it easy to compare columns and keeps the result in sorted order.
2. Take each row *x* of the first matrix and each row *y* of the transposed second matrix, and compare the **column values** of their entries.
3. Wherever the column values match, **multiply** the two values.
4. **Add** all such products. If the total is non-zero, store it as result[x][y].

6. Applications of Arrays

Application	How arrays are used
Storing lists of data	Student marks, salaries, daily temperatures, product prices.
Sparse matrices	Triplet tables keep only the non-zero entries to save memory.
Building other structures	Stacks, queues, heaps, hash tables and adjacency matrices for graphs are often array-based.
Searching and sorting	Linear search, binary search, bubble, selection and quick sort all operate on arrays.
Polynomials	The index represents the power and the stored value is the coefficient.
Lookup tables	Direct access by index suits names of weekdays or months and character-frequency counts.
Images and graphics	An image is a 2-D array of pixels, each holding a colour value.
Games and boards	Chess, tic-tac-toe, sudoku and mazes use 2-D arrays for the board.
Tables and spreadsheets	Rows and columns of tabular data behave like a 2-D array.

7. String Manipulation Using Arrays

A **string** is a finite sequence of characters. In data-structure terms it is stored as a one-dimensional character array in contiguous memory. In **C**, the end of the string is marked by the **null character** `'\0'`, so a string of n characters needs an array of size **n + 1**.

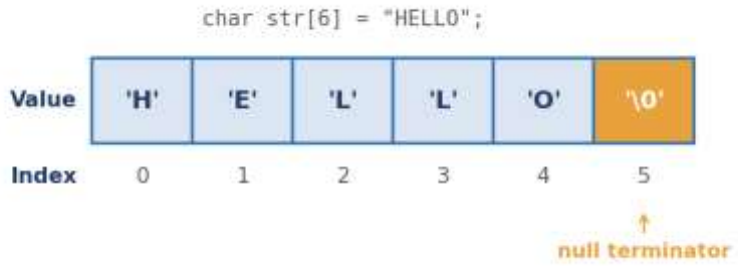


Figure 11: *Memory layout of the C string "HELLO"*

In Java, `String` is a built-in class rather than a raw character array, so the null terminator belongs to the C view of strings.

Basic string operations

Operation	Idea
Length	Count characters until the null character is reached.
Copy	Copy each character from the source to the destination, then add the null character.
Concatenation	Append the second string after the last character of the first.
Reverse	Swap characters from the two ends, moving towards the middle.
Comparison	Compare character by character; stop at the first difference or at the null character.
Substring	Extract `len` characters starting from position `pos`.

8. Quick Revision

Key points to remember

• A data structure organises data for efficient use; it is either **primitive** or **non-primitive**.

• Linear structures: array, stack (LIFO), queue (FIFO), linked list. Non-linear: tree, graph.

• An algorithm must be finite, unambiguous and independent of any language. Compare algorithms by time and space complexity.

• Arrays store same-type items contiguously; index access is O(1), while insertion and deletion are O(n) because of shifting.

• Java arrays live on the heap; the reference variable lives on the stack.

• A sparse matrix has mostly zeros. Triplet form stores (row, column, value); the linked-list form uses nodes with a next pointer.

• Matrix addition needs equal dimensions; multiplication needs columns of A = rows of B.

Practice questions

1. Define a data structure and explain how primitive and non-primitive structures differ.
2. Distinguish between linear and non-linear data structures with two examples each.
3. List and explain the characteristics of an algorithm.
4. Write a Java program to insert an element at a given position in an array and explain the shifting.
5. Compare static and dynamic memory allocation. How does Java allocate arrays?
6. Represent the following matrix in triplet form and find its transpose.
7. Explain the steps for adding two sparse matrices stored in triplet form.
8. State five applications of arrays.